

SVD & PCA

Low-rank approximation and principal directions

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Low-rank updates for large models

Fine-tuning moves every weight

A pretrained 7-billion-parameter language model already knows language. You want it to write *your* hospital's radiology reports.

- **Fine-tuning** = nudge the weights: every matrix $W \rightarrow W + \Delta W$
- Naively, ΔW is as big as the model: **7 billion** new numbers to learn, store, and ship
- One fine-tune per task, per clinic, per customer... each a full model copy

Question of the day: does the *update* really need all 7,000,000,000 numbers?

LoRA's bet: the update is low-rank

I INTERACTIVE

One weight matrix inside such a model: $W \in \mathbb{R}^{4096 \times 4096}$ — **16,777,216** numbers.

LoRA (Hu et al., 2021) learns the update as a product of two skinny matrices:

$$\Delta W = \underbrace{B}_{4096 \times 8} \underbrace{A}_{8 \times 4096} \rightarrow 2 \cdot 4096 \cdot 8 = 65,536 \text{ numbers} — 0.4\%, \text{ a } 256 \times \text{ saving}$$

- $\Delta W = BA$ has **rank at most 8** — a sum of 8 outer products (L4)
- GPT-3 (175B): **~10,000×** fewer trainable parameters, matching accuracy

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/lora-adapter

Slide the adapter rank r ; watch the parameter meter collapse.

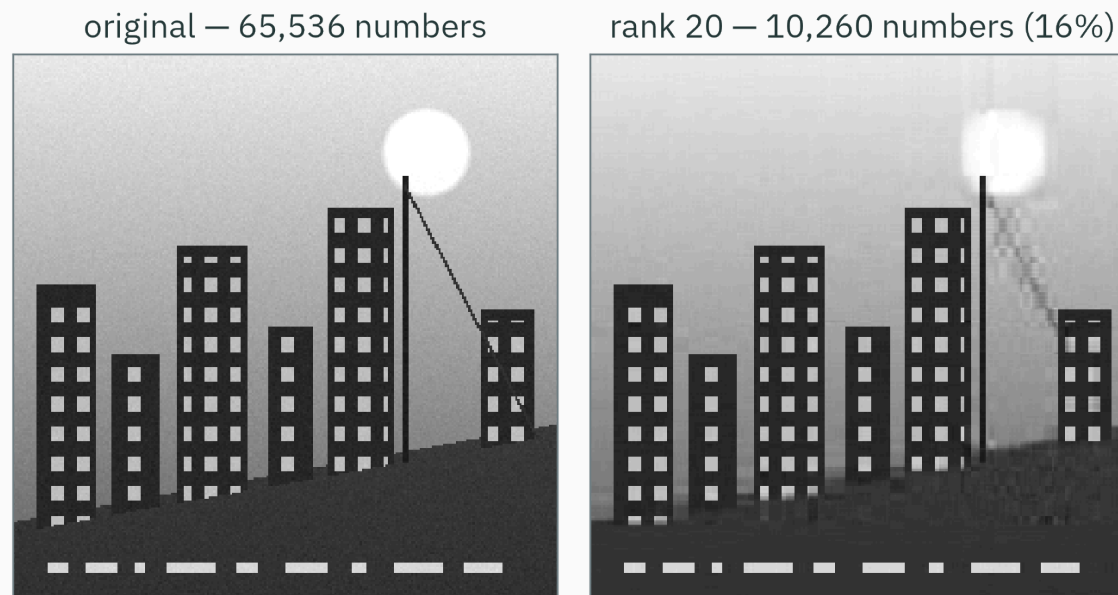
Why would low-rank be enough?

That is a **mathematical** question, and this lecture is the answer:

1. Every matrix admits two orthonormal coordinate systems with an axis scaling between them (the SVD)
2. The singular values come sorted; when they decay quickly, a few directions dominate
3. Keeping the top k is **provably the best** rank- k approximation (Eckart–Young)

Training LoRA is covered in the deep-learning course (ES 667). Today we study the low-rank factorization and test it on an image where the approximation error is directly visible.

The same trick, visible: image compression



A 256×256 image *is* a matrix. Right: the same matrix, rebuilt from its top 20 “stretches”.

By the end of class you will compute exactly *which* 16% to keep — and prove nothing better exists at that budget.

Learning outcomes

By the end of this lecture you will be able to:

1. Describe what **any** matrix does using orthonormal input/output coordinates and axis scaling, and read singular values as ellipse semi-axes.
2. Compute a full **2×2 SVD by hand** via $A^T A$, and verify it multiplies back.
3. Expand a matrix into **rank-1 layers**, ranked by energy σ_i^2 .
4. State **Eckart–Young** and use the σ -tail to predict compression error.
5. Derive **PCA** as best-fit axes / max-variance directions, and run it on real data.
6. Show **PCA = SVD of the centered data matrix**, and pick the right route in code.

**Every matrix: choose
coordinates · scale · place the
output**

L5 gave us eigenvectors — and left a gap

L5: eigenvectors are the directions a matrix *can't turn* — and symmetric matrices got the dream: $S = Q\Lambda Q^\top$, real stretches along perpendicular axes.

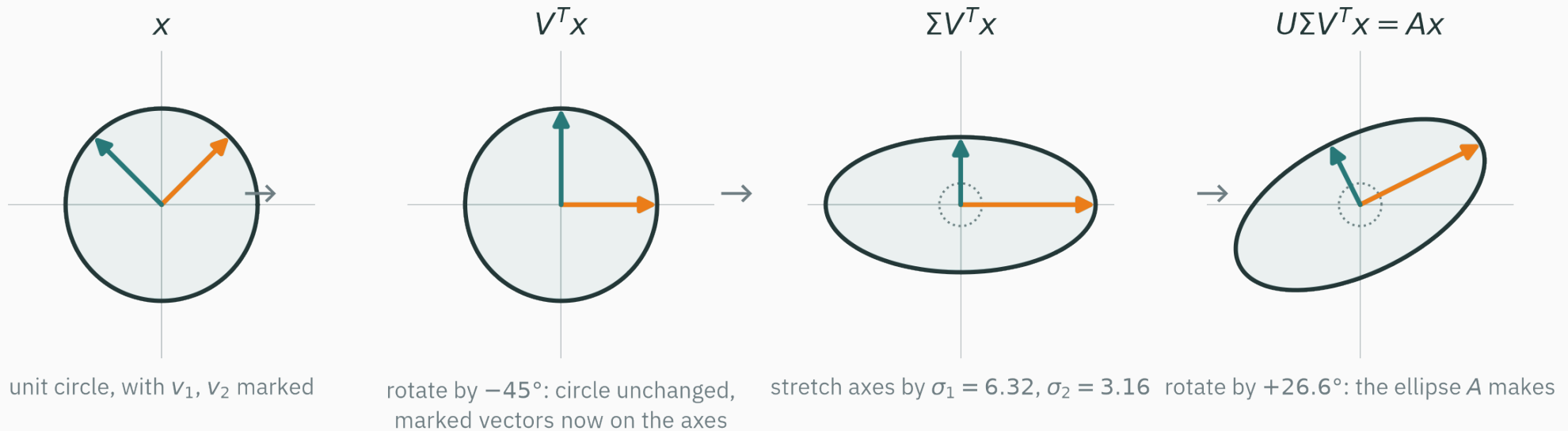
But the dream has fine print:

- A plain square matrix may have **complex** eigenvalues (a rotation turns *every* direction)
- And most matrices in AI are not even square: a dataset is 1797×64 , an MLP weight is 4096×11008 — “ $Av = \lambda v$ ” doesn't typecheck when input and output live in different spaces

Today's theorem: **every matrix — square or not — factors perfectly. The price: two orthonormal bases instead of one.**

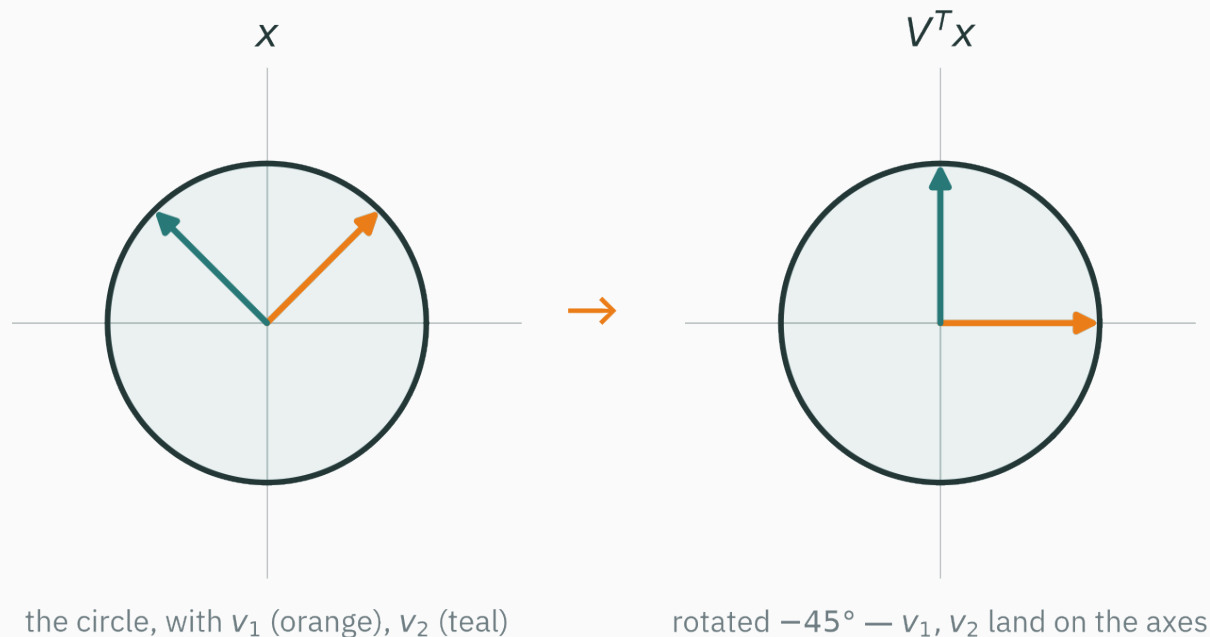
What a matrix does to the unit circle

Feed every unit vector through $A = \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix}$ and watch:



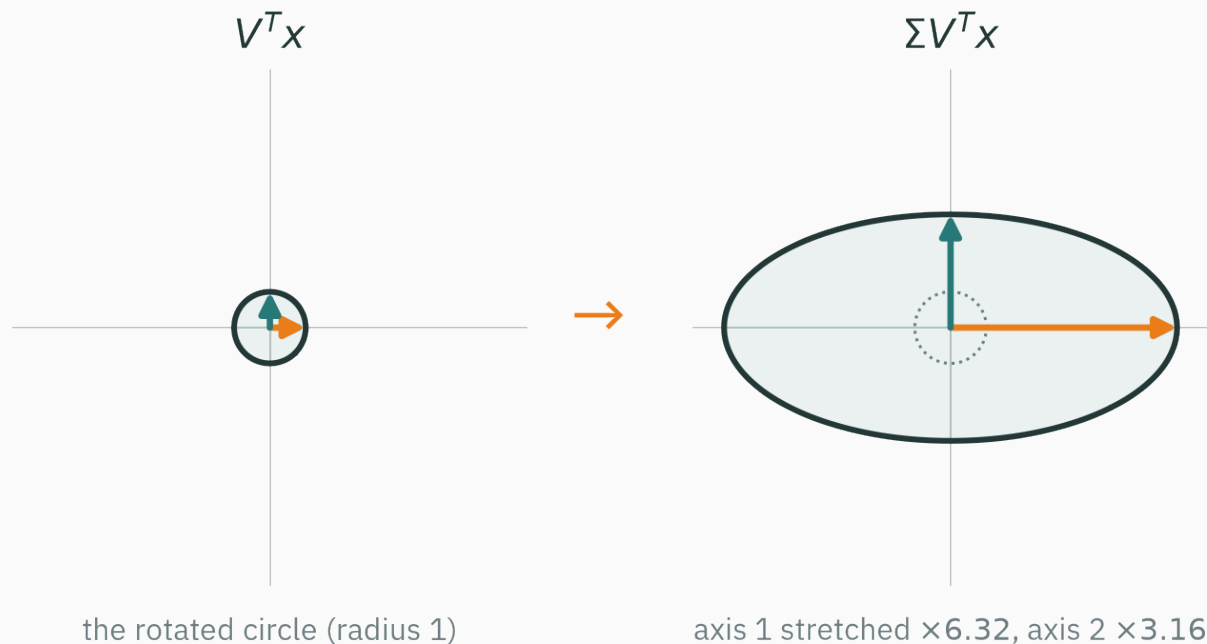
A circle in, an **ellipse** out — always. The three arrows between the panels are the three machines hiding inside A .

Step 1 · V^T rotates the input into position



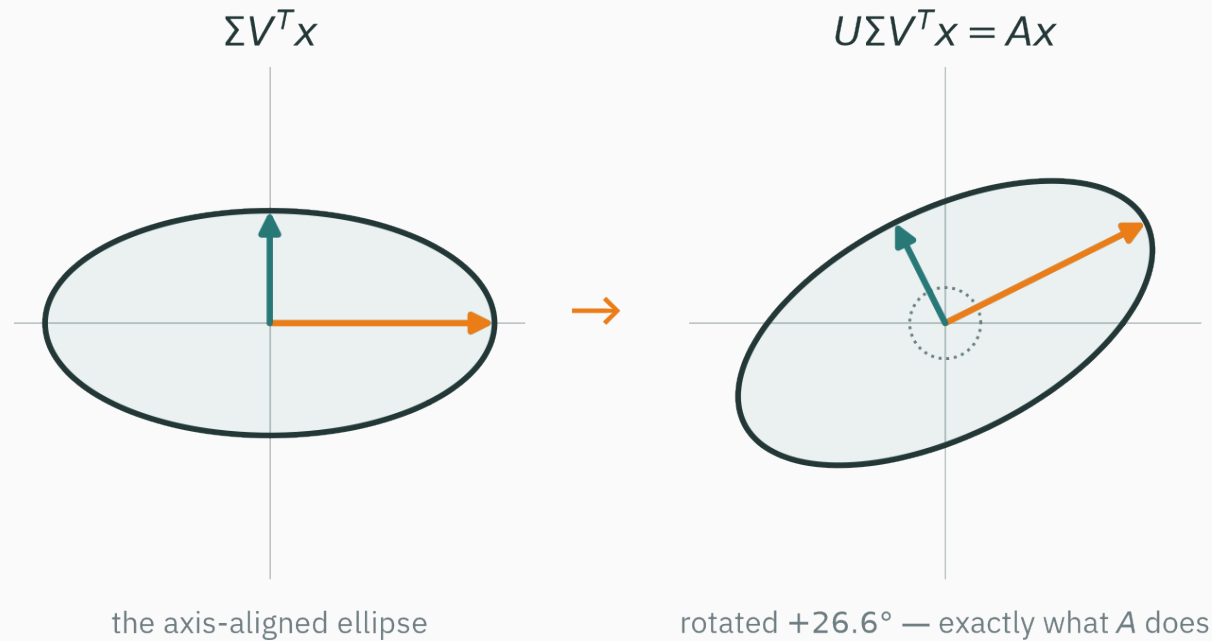
- A rotation: the circle doesn't change — but two special directions v_1, v_2 (the **right singular vectors**) land exactly on the coordinate axes
- Perpendicular before, perpendicular after — rotations preserve angles

Step 2 · Σ stretches along the axes



- A pure axis-aligned stretch: $\times 6.32$ horizontally, $\times 3.16$ vertically
- The **only** place sizes change — all of A 's “action”, concentrated in two positive numbers $\sigma_1 \geq \sigma_2$: the **singular values**

Step 3 · U rotates to the output pose



- A final rotation sets the ellipse's tilt: the axes land on u_1, u_2 — the **left singular vectors**
- Net effect: exactly what A does — rebuilt from a rotation, a stretch, and a rotation

The claim, in symbols

$$A = \underbrace{U}_{\text{rotate (output)}} \underbrace{\Sigma}_{\text{stretch}} \underbrace{V^T}_{\text{rotate (input)}}$$

read right-to-left, like function composition (L4).

- U, V : **orthonormal** columns (perpendicular unit vectors). In the square full SVD they are rotations/reflections; in a thin rectangular SVD they are orthonormal coordinate frames.
- Σ : **diagonal**, entries $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$

Singular value decomposition: every matrix is an orthogonal change of input coordinates, an axis scaling (with discarded null directions), and an orthogonal placement in the output space.

Rectangular is fine — the shapes

For $A \in \mathbb{R}^{m \times n}$ with rank r , the (thin) SVD reads

$$\underbrace{A}_{m \times n} = \underbrace{U}_{m \times r} \underbrace{\Sigma}_{r \times r} \underbrace{V^T}_{r \times n}$$

- V^T expresses the input in right-singular coordinates and retains the r active components; U places those components in the **output** space $\mathbb{R}^m$
- Σ stretches r surviving directions and drops the rest

The “full” SVD pads U to $m \times m$ and V to $n \times n$ with basis vectors for the leftover space. ML code wants the thin one: `np.linalg.svd(A, full_matrices=False)`.

Reading Σ : semi-axes, importance, rank

The singular values are the **semi-axes of the output ellipse** — and they come sorted:

- σ_1 : the largest stretch A applies to any unit vector (its “amplification record”)
- $\sigma_i = 0$: that direction is flattened away entirely — the ellipse loses a dimension
- **$\text{rank}(A)$ = number of nonzero σ_i** — L4’s “how much of space survives”, now a countable quantity

Eigenvalues can be negative or complex. Singular values are **always real and ≥ 0** — signs and turns are absorbed by U and V .

A 2×2 SVD, by hand

Where do v and Σ come from? Ask $A^\top A$

How strongly does A stretch a unit vector v ? Square it:

$$\|Av\|^2 = (Av)^\top (Av) = v^\top (A^\top A)v$$

- $A^\top A$ is **symmetric** — L5's spectral theorem applies: real eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$, **orthonormal** eigenvectors
- Along an eigenvector: $\|Av_i\|^2 = \lambda_i$ — so the stretch there is $\sqrt{\lambda_i}$

v_i = eigenvectors of $A^\top A$, and $\sigma_i = \sqrt{\lambda_i}$. L5's spectral theorem is the engine inside the SVD.

Worked 2×2 · the eigenvalues of $A^T A$

$$A = \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} \rightarrow A^T A = \begin{pmatrix} 5 & 0 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} = \begin{pmatrix} 25 & 15 \\ 15 & 25 \end{pmatrix}$$

Characteristic polynomial (L5, on a symmetric 2×2):

$$(25 - \lambda)^2 - 15^2 = 0 \rightarrow 25 - \lambda = \pm 15 \rightarrow \lambda_1 = 40, \quad \lambda_2 = 10$$

- Singular values: $\sigma_1 = \sqrt{40} = 2\sqrt{10} \approx 6.32$, $\sigma_2 = \sqrt{10} \approx 3.16$
- Eigenvectors: λ_1 : $-15v_x + 15v_y = 0 \rightarrow v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$; perpendicular: $v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$ — the $\pm 45^\circ$ directions in the step-1 picture

The left side: where the axes land

u_i = the unit vector that v_i is sent to. Since $\|Av_i\| = \sigma_i$, normalize:

$$u_1 = \frac{Av_1}{\sigma_1} = \frac{1}{2\sqrt{10}} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 8 \\ 4 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

$$u_2 = \frac{Av_2}{\sigma_2} = \frac{1}{\sqrt{10}} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} -2 \\ 4 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}$$

- Check: $u_1 \cdot u_2 = (-2 + 2)/5 = 0$ — automatically perpendicular
- $U = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & -1 \\ 1 & 2 \end{pmatrix}$ is a rotation by $\arctan(1/2) \approx 26.6^\circ$ — the tilt in the step-3 picture

Multiply back — it really is A

Assemble the three machines. First the stretch applied to the rotated frame:

$$\Sigma V^{\top} = \begin{pmatrix} 2\sqrt{10} & 0 \\ 0 & \sqrt{10} \end{pmatrix} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix} = \sqrt{5} \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix}$$

then the output rotation — the $\sqrt{5}$'s cancel and the arithmetic goes integer:

$$U(\Sigma V^{\top}) = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & -1 \\ 1 & 2 \end{pmatrix} \cdot \sqrt{5} \begin{pmatrix} 2 & 2 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 4+1 & 4-1 \\ 2-2 & 2+2 \end{pmatrix} = \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} \checkmark$$

$U\Sigma V^{\top} = A$, exactly. Rotate · stretch · rotate is not a metaphor — it's a factorization you can verify by hand.

Two sanity checks that generalize

Areas. A rotation preserves area; the stretch scales it by $\sigma_1 \sigma_2$:

$$\sigma_1 \sigma_2 = 2\sqrt{10} \cdot \sqrt{10} = 20 = 5 \cdot 4 - 3 \cdot 0 = \det A \quad \checkmark$$

Total content. Sum the squares of all entries (the *Frobenius norm* $\|A\|_F^2$):

$$5^2 + 3^2 + 0^2 + 4^2 = 50 = 40 + 10 = \sigma_1^2 + \sigma_2^2 \quad \checkmark$$

Both are theorems, not luck: $|\det A| = \prod_i \sigma_i$, and $\|A\|_F^2 = \sum_i \sigma_i^2$ — rotations change neither areas nor lengths, so only Σ can.

The machine agrees

```
>>> A = np.array([[5., 3.], [0., 4.]])
>>> U, S, Vt = np.linalg.svd(A)
>>> S
array([6.3246, 3.1623])          # 2*sqrt(10), sqrt(10)
>>> U
array([[ 0.8944, -0.4472],        # (2,1)/sqrt(5) in column 1
       [ 0.4472,  0.8944]])
>>> np.allclose(U @ np.diag(S) @ Vt, A)
True
```

The SVD is unique up to **paired sign flips**: $(u_i, v_i) \rightarrow (-u_i, -v_i)$ is equally valid, so your hand solution may differ from NumPy's by minus signs. Σ (sorted descending) is always the same.

A is 2×2 with $\sigma_1 = 10$ and $\sigma_2 = 0.01$. Which conclusion is *forced*?

A. A maps the unit circle to a circle of radius ≈ 10

B. A maps the unit circle to a near-1-D sliver — A is numerically almost rank 1

C. $A \approx 10I$: roughly “multiply by 10”

D. Nothing can be said without knowing U and V

Correct: B — a 10×0.01 sliver — numerically almost rank 1

Why: U and V only *pose* the ellipse; its shape is fixed by Σ alone. With $\sigma_2 \approx 0$ the second axis is flattened: outputs hug a single line, one rank-1 layer carries $10^2 / (10^2 + 0.01^2) \approx 99.9999\%$ of the energy. (A and C would need $\sigma_1 \approx \sigma_2$.)

Keep only the big stretches

A matrix is a stack of rank-1 layers

Multiply $U\Sigma V^\top$ out column-by-row and the SVD becomes a **sum**:

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \quad \text{— each } \mathbf{u}_i \mathbf{v}_i^\top \text{ a rank-1 layer (L4's } \star \text{ fact), weighted by } \sigma_i$$

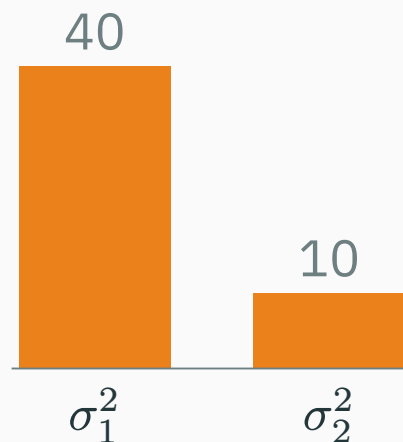
Our worked matrix, split into its two layers:

$$\begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} = \underbrace{\begin{pmatrix} 4 & 4 \\ 2 & 2 \end{pmatrix}}_{\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top} + \underbrace{\begin{pmatrix} 1 & -1 \\ -2 & 2 \end{pmatrix}}_{\sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top}$$

The SVD ships every matrix **pre-sliced into layers, sorted by importance.**
Now: what if we keep only the top of the stack?

“Importance” made precise: energy

From the sanity check: $\|A\|_F^2 = \sum_i \sigma_i^2$. So layer i **owns** σ_i^2 of the matrix’s total squared content — its **energy**:



For our 2×2: an **80% / 20%** split — the first layer already *is* most of the matrix. Real data matrices are far more lopsided, as you’re about to see.

Truncate the stack: the storage arithmetic

Keep the top k layers and discard the rest:

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \rightarrow \text{store } k \text{ of the } \mathbf{u}\text{s, } k \text{ of the } \mathbf{v}\text{s, } k \text{ } \sigma\text{s} = k(m + n + 1) \text{ numbers}$$

For a 256×256 image (65,536 numbers): rank k costs $513k$.

- $k = 20$: 10,260 numbers — **15.7%** of the original, a 6.4× saving
- LoRA's 4096×4096 , $k = 8$: 65,536 numbers — **0.4%**. Same formula.

But is blunt truncation a *good* approximation — or just a cheap one?

Eckart–Young: truncation is optimal

Among all matrices B of rank $\leq k$, the truncated SVD A_k is the one closest to A : $\min_{\text{rank}(B) \leq k} \|A - B\|_F = \|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_r^2}$

- The error is exactly the **energy left in the tail** — read your loss straight off the σ 's, before committing to anything
- No cleverer rank- k matrix exists. Searching is pointless; **truncating is already optimal**
- This is the course's first “**learning = approximation**” theorem: replace a big object by the best small one and *know* what you lost. (Also true in the spectral norm — not shown here.)

Which direction does A amplify most?

Let's earn the $k = 1$ case. Start by finding the unit direction that A amplifies most:

$$\max_{\|v\|=1} \|Av\| = ?$$

Claim: the maximizer is v_1 — the top right singular vector — with value σ_1 .

Strategy: don't fight in standard coordinates. **Change to the basis the SVD hands you.**

$v_1, \dots, v_n$ are orthonormal (spectral theorem, L5), so any unit v is

$$v = c_1 v_1 + \dots + c_n v_n, \quad \text{with } \sum_i c_i^2 = 1$$

Apply A , using $Av_i = \sigma_i u_i$:

$$Av = \sum_i c_i \sigma_i u_i \quad \rightarrow \quad \|Av\|^2 = \sum_i \sigma_i^2 c_i^2$$

(the cross terms die because the u_i are orthonormal too — both bases pull their weight)

$\|Av\|^2 = \sum_i \sigma_i^2 c_i^2$ is a **weighted average** of the σ_i^2 with weights c_i^2 summing to 1:

$$\|Av\|^2 \leq \sigma_1^2 \cdot \sum_i c_i^2 = \sigma_1^2, \quad \text{equality} \leftrightarrow \text{all weight on } c_1 \leftrightarrow v = \pm v_1$$

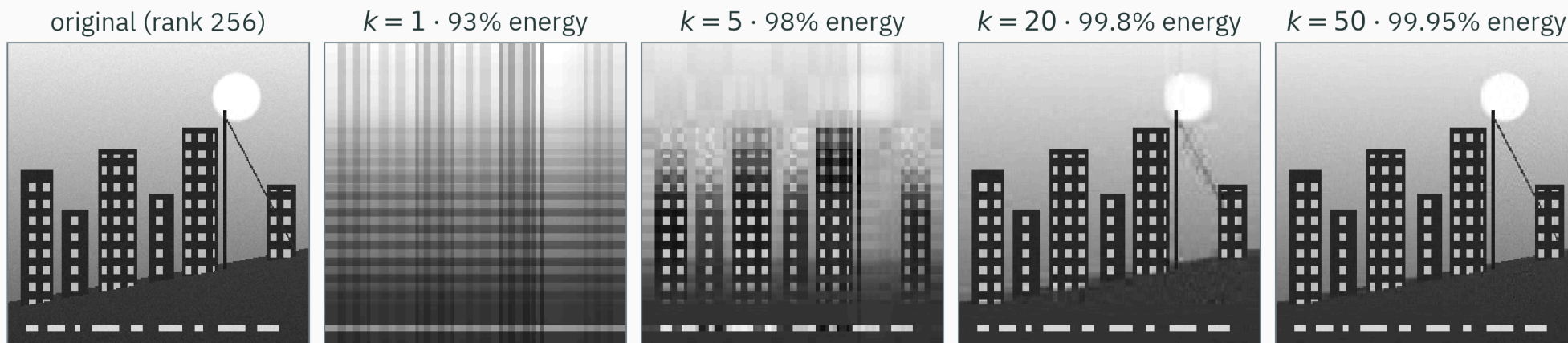
- So the strongest direction is v_1 , amplified σ_1 -fold onto u_1 — and the best rank-1 layer is $A_1 = \sigma_1 u_1 v_1^\top$, leaving $\text{error}^2 = \sigma_2^2 + \dots + \sigma_r^2$
- Rank k : subtract the layer, repeat on the remainder — greedy, and here **greedy is optimal**

The honest gap: we showed v_1 wins the amplification contest; that *no other* rank-1 matrix removes more energy is the full Eckart–Young proof — MML §4.5.

The demo: image compression

One image, four budgets

A = a 256×256 grayscale image. Compute `np.linalg.svd(A)`, keep k layers:



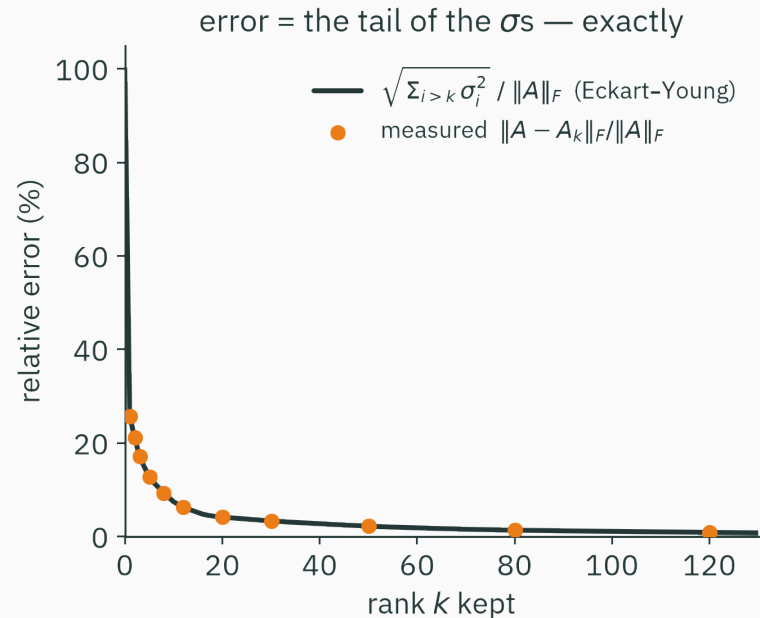
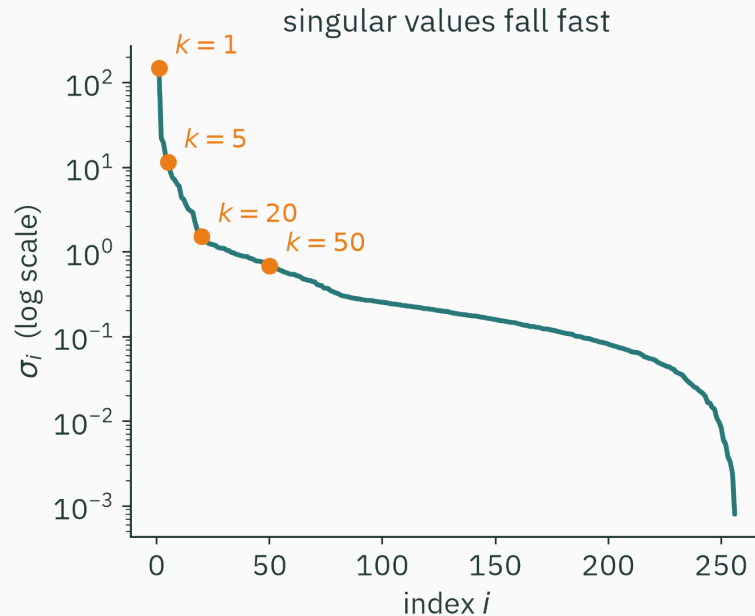
$k = 1$ is literally one column times one row — yet the sky gradient is already there. By $k = 20$ you'd hand this to a human.

The demo's ledger

rank k	numbers stored	of 65,536	saving	energy kept	rel. error
1	513	0.8%	128×	93.4%	25.7%
5	2,565	3.9%	26×	98.4%	12.8%
20	10,260	15.7%	6.4×	99.8%	4.2%
50	25,650	39.1%	2.6×	~100%	2.2%

- “Energy kept” = $\sum_{i \leq k} \sigma_i^2 / \sum_i \sigma_i^2$; “rel. error” = $\|A - A_k\|_F / \|A\|_F$
- $k = 1$ keeps 93% of the squared Frobenius norm — one rank-1 layer already captures the dominant light–dark pattern

Why it works: the spectrum falls off a cliff



- Left: σ_i on a log scale — **structured** images spend their energy in few layers
- Right: measured error (dots) lands *exactly* on the Eckart-Young tail $\sqrt{\sum_{i>k} \sigma_i^2}$ (line) — the theorem, photographed

LoRA uses a low-rank parameter update

Replace “image” by “weight update” and rerun the argument:

- LoRA’s bet: the *useful* change to a weight matrix concentrates its energy in a few directions — like the image, a cliff-shaped spectrum
- Then Eckart–Young says a rank- r update loses almost nothing — while the storage line reads $k(m + n)$ instead of mn : **0.4%** of the parameters for $r = 8$
- Fine-tune = ship the tiny B, A pair; the 7B base model stays frozen, shared by every task

The same low-rank approximation applies to image compression and model updates; next we apply it to a centered dataset.

Compression in six lines

```
img = plt.imread("campus.png").mean(axis=2)      # (256, 256) grayscale
U, S, Vt = np.linalg.svd(img, full_matrices=False)

k = 20
img_k = U[:, :k] * S[:k] @ Vt[:k]               # broadcast: U_k diag(S_k) V_k^T
np.linalg.norm(img - img_k) / np.linalg.norm(img)  # 0.042 — as the table said
```

- $U[:, :k] * S[:k]$ scales column i by σ_i — the standard idiom for $U_k \Sigma_k$
- Tutorial 3: you'll run this on your own photo and plot the whole error-vs- k curve

A is 4×4 with singular values $\sigma = (6, 3, 2, 1)$. You keep A_2 , the best rank-2 approximation. What is $\|A - A_2\|_F$?

A. 2**B.** 3**C.** $\sqrt{5}$ **D.** $\sqrt{14}$

Correct: C — $\sqrt{5} \approx 2.24$

Why: Eckart–Young: the error is the tail in *quadrature*, $\sqrt{\sigma_3^2 + \sigma_4^2} = \sqrt{4 + 1}$ — discarded layers are orthogonal, so their energies add like Pythagoras. A forgets σ_4 ; B adds instead of squaring; D starts the tail at σ_2 , which A_2 still keeps.

PCA: the best low-dimensional summary of a dataset

Datasets are wide; structure is thin

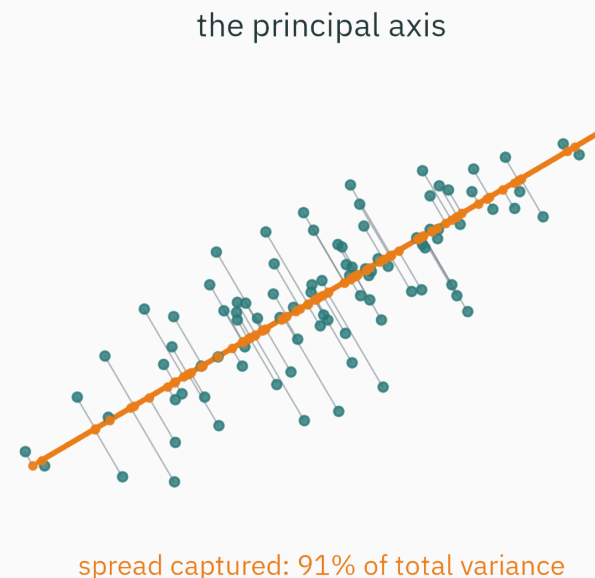
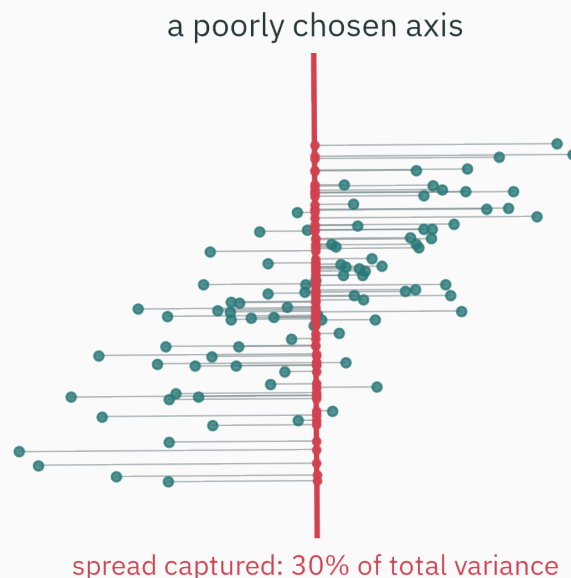
Stack n data points as the rows of a matrix $X \in \mathbb{R}^{n \times d}$:

- 1797 handwritten digits $\times$ 64 pixel values
- a smartwatch: millions of timestamps $\times$ dozens of sensor channels
- L3's word embeddings: vocabulary $\times$ 300 coordinates
- Coordinates are **redundant**: neighboring pixels agree, sensors co-move, features correlate. The cloud is d -dimensional on paper but hugs a **much flatter** subspace

The matrix most worth compressing is the dataset itself. Finding its best flat summary has a name: **PCA.**

Which axis keeps the most variance?

Project a centered 2-D cloud onto one line — which line loses least?



Same 90 points: the left axis keeps 30% of the spread, the **principal axis** 91%.
PCA is the search for that winning direction (then the runner-up, and so on).

Two definitions, one direction

For centered data, split each point against a candidate unit axis w — Pythagoras:

$$\underbrace{\|x_i\|^2}_{\text{fixed}} = \underbrace{(w^\top x_i)^2}_{\text{spread along } w} + \underbrace{\|x_i - (w^\top x_i)w\|^2}_{\text{residual}^2}$$

Sum over the cloud: the left side is fixed, so **max variance** $\leftrightarrow$ **min reconstruction error**. Best-fit line and max-spread direction are the *same* axis.

Total spread along w is $\|Xw\|^2$ and is maximized by the top right singular vector v_1 of X . This is the first principal direction.

The PCA recipe

Given raw data $X \in \mathbb{R}^{n \times d}$, want k axes:

1. **Center**: $X \leftarrow X - \bar{x}^\top$ (subtract the mean of each column)
2. **Covariance**: $C = \frac{1}{n} X^\top X \in \mathbb{R}^{d \times d}$ — entry (a, b) : how features a, b co-vary
3. **Eigendecompose** (symmetric! L5): $C = V \Lambda V^\top$, $\lambda_1 \geq \dots \geq \lambda_d \geq 0$
4. **Project**: $z_i = V_k^\top x_i \in \mathbb{R}^k$ — the point in “principal coordinates”

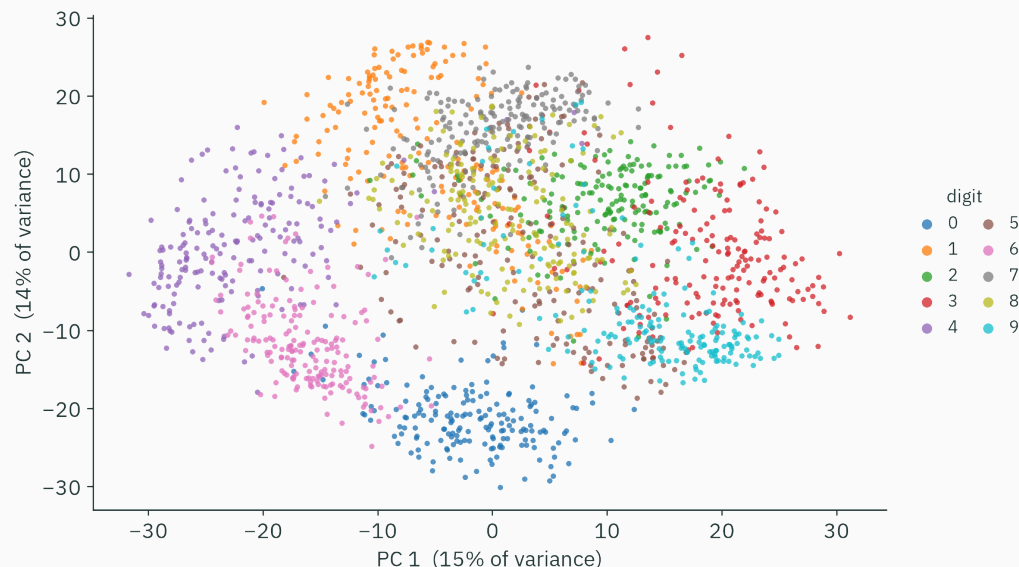
λ_j = variance captured by axis j ; reconstruct with $\hat{x}_i = V_k z_i + \bar{x}$.

Convention: we divide by n ; statisticians and sklearn use $n - 1$. Same axes either way — only the λ scale shifts.

The recipe in NumPy

```
Xc = X - X.mean(axis=0)      # 1. center          (n, d)
C = Xc.T @ Xc / len(Xc)      # 2. covariance       (d, d)
lam, V = np.linalg.eigh(C)    # 3. eigh -> ASCENDING order...
lam, V = lam[::-1], V[:, ::-1] # ...flip: biggest first
Z = Xc @ V[:, :k]            # 4. project          (n, k)
```

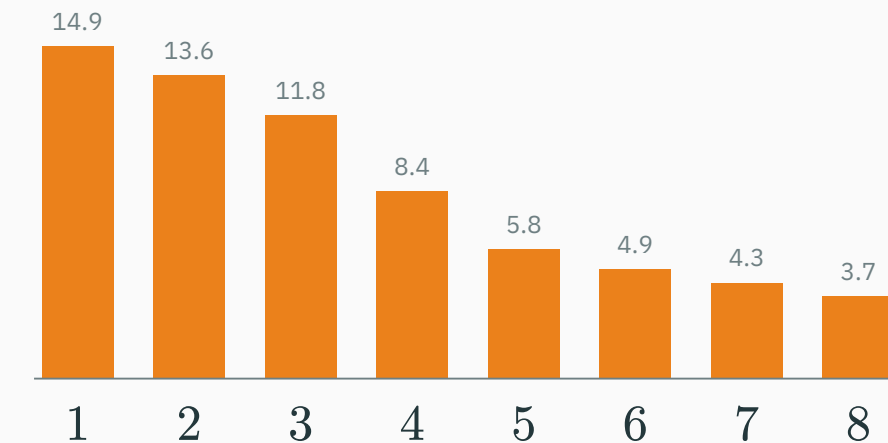
`eigh` returns eigenvalues **smallest-first** — forgetting the flip silently hands you the k *least* informative axes. A classic, painful bug.



- Two axes keep 28.5% of the variance — yet 0s, 6s, 4s already form islands
- PCA never saw the labels; the colors were painted on *afterwards* — the structure was in the pixels all along

A real dataset's spectrum

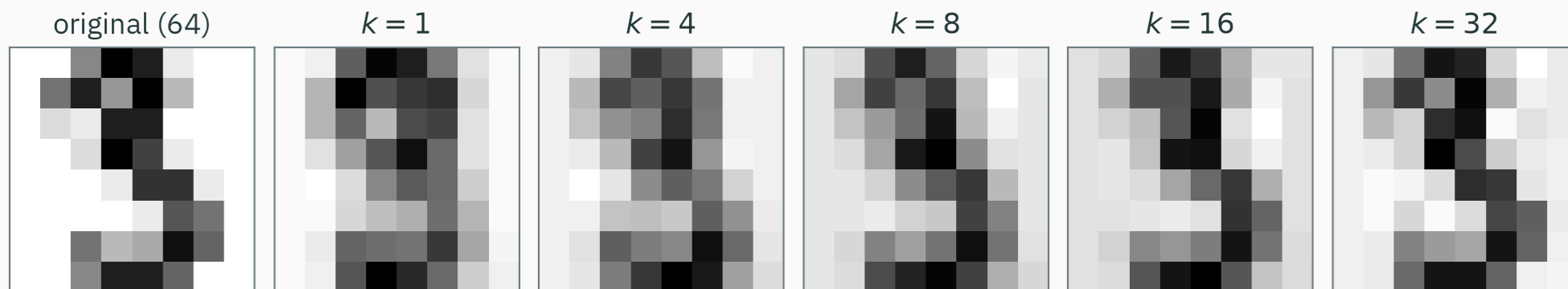
Variance share of the top principal axes (digits, $d = 64$):



percent of total variance, by principal-axis index

The image's cliff again, in dataset form: **21 of 64** axes already carry 90% of the variance.

Reconstruct one digit from its first k principal coordinates ($\hat{x} = V_k z + \bar{x}$):



- $k = 1$: a generic average-ish digit; $k = 8$: recognizably a 3; $k = 32$: essentially perfect from **half** the numbers
- This is Eckart–Young acting on a *data* matrix — compression and PCA are one operation

**The bridge: PCA is the SVD of
centered data**

Two routes, one set of axes

Let X be **centered**, with SVD $X = U\Sigma V^\top$. Feed it into the covariance:

$$C = \frac{1}{n}X^\top X = \frac{1}{n}(U\Sigma V^\top)^\top (U\Sigma V^\top) = \frac{1}{n}V\Sigma U^\top U\Sigma V^\top = V\left(\frac{\Sigma^2}{n}\right)V^\top$$

That *is* an eigendecomposition of C — orthonormal V , diagonal middle. Read it off:

- principal axes = **right singular vectors** of X ; variances $\lambda_i = \sigma_i^2/n$
- and the mirror fact: $XX^\top = U\Sigma^2U^\top$ — the $n \times n$ point-similarity matrix shares the same spectrum

PCA = SVD of the centered data matrix. Covariance eigenvectors and the right singular vectors of centered data are the same axes.

The bridge, checked in code

```
Xc = X - X.mean(axis=0)          # digits: (1797, 64)
n = len(Xc)

lam, V = np.linalg.eigh(Xc.T @ Xc / n)    # route 1: covariance
lam, V = lam[::-1], V[:, ::-1]
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)  # route 2: SVD

np.allclose(S**2 / n, lam)              # True
np.allclose(np.abs(Vt[:20]), np.abs(V.T[:20]))    # True (signs may flip)
```

Why compare only the top 20? Some digit pixels are constant $\rightarrow$ variance 0 $\rightarrow$ those tail axes are arbitrary in *both* routes. Zero-eigenvalue directions have no preferred basis.

Centering is the fine print

Skip step 1 and “PCA” quietly answers a different question:

- Three points hugging (100, 100): (99, 101), (100, 100), (101, 99)
- **Uncentered** top singular direction: (0.707, 0.707) with $\sigma_1 \approx 245$ — it points **at the mean**, because reaching the cloud from the origin dwarfs the spread
- **Centered**: (0.707, -0.707) — the direction the data actually varies

SVD of raw data $\approx$ “where is the cloud?” SVD of centered data = “which way does it vary?” Only the second is PCA. Forgetting to center is the field’s favorite silent bug.

In practice, take the SVD route

Both routes are correct mathematics. Numerically they are not equals:

- **Memory**: for $d = 10^5$ features, C is $10^5 \times 10^5 = 10^{10}$ entries you never needed; SVD works on X directly
- **Precision**: forming $A^\top A$ squares the singular values — $\sigma_i = 10^{-8}$ becomes $\lambda_i = 10^{-16}$, close to float64 ϵ . The numerical example reports a tail eigenvalue of -0.0 , a rounding artifact rather than negative variance
- direct and randomized-SVD PCA solvers work on X_c without forming C ; some libraries also offer a covariance-eigensolver when that shape is favourable

Think in covariance eigenvectors; *compute* with the SVD.

Wrap-up

Practice problems

Try on paper; verify in the Tutorial 3 notebook.

1. Hand-SVD $\begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}$: find σ_1, σ_2 and both bases. What are the “rotations” here?
2. The SVD of $\begin{pmatrix} 3 & 0 \\ 0 & -2 \end{pmatrix}$ has $\sigma = (3, 2)$ — a diagonal matrix that is *not* its own Σ . Where did the minus sign and the ordering go?
3. $\sigma = (10, 6, 3, 1)$: compute the *relative* Frobenius error of A_2 . ($\approx 26\%$.)
4. A 512×512 image: what is the largest k that still compresses at least $4\times$?
5. Compute the top singular direction of the three-point cloud
(99, 101), (100, 100), (101, 99) — uncentered, then centered. Explain the flip.
6. Prove $\|A\|_F^2 = \sum_i \sigma_i^2$. (Hint: rotations preserve lengths — or take $\text{tr}(A^\top A)$.)



Too big to SVD? Randomize

★ OPTIONAL

Exact SVD costs $O(mn \min(m, n))$ — hopeless for a $10^6 \times 10^5$ matrix. The modern fix is **randomized SVD**:

1. Sketch: $Y = A\Omega$ with a random $n \times (k + 10)$ Gaussian Ω — a few random probes almost surely capture the span of the top layers
2. Orthonormalize Y (QR), project A into that thin subspace, SVD the small matrix
3. Guarantee: error close to the optimal σ_{k+1} , with high probability — at a fraction of the cost

Halko–Martinson–Tropp (2011). fast.ai's free *Computational Linear Algebra* course builds it in NumPy — and sklearn's `PCA(svd_solver="randomized")` is what actually runs on large data.

Lecture 6 — summary

- **SVD**: every matrix is $U\Sigma V^\top$ — orthonormal input/output coordinates with axis scaling; σ_i = ellipse semi-axes; rank = surviving axes; computed by hand via the eigenvectors of $A^\top A$ (L5's spectral theorem).
- **Layers**: $A = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, importance-sorted; energy $\|A\|_F^2 = \sum_i \sigma_i^2$.
- **Eckart–Young**: truncating at k is the *best* rank- k approximation; error = the σ -tail. One theorem behind image compression **and** LoRA's 0.4% fine-tuning.
- **PCA**: best-fit axes = max-variance directions = eigenvectors of the covariance.
- **The bridge**: PCA is the SVD of *centered* data ($\lambda_i = \sigma_i^2/n$) — compute via SVD.

Read before L7 — MML Ch 4.4–4.5 (SVD) and Ch 10 (PCA); Solomon Ch 7.

Tutorial 3 this week: SVD compression on your own photo + PCA on digits. **Quiz 1** covers Module 1 (L1–L6) — the worked 2×2 here is exam-shaped.

Every matrix has orthonormal input and output directions connected by singular-value scaling.

Module 2 begins. Next: **Univariate Calculus & Taylor Series** — we stop rearranging space and start measuring change.